

Typst + GitHub · Ubuntu 24.04.2

XCPC Typst 代码模板库

使用手册

添加和修改代码片段 · 组织目录与章节 · GitHub 多设备同步 · 字体与排版设置

本手册配套 `xcpc-typst-codebook-starter.zip` 使用。所有命令默认在项目根目录执行。

版本 1.0 · 2026-08-08

目录

1	先认识这套模板	2
1.1	最常用的工作流	2
2	添加和修改代码片段	3
2.1	添加一个新算法：以 Boruvka 为例	3
2.2	修改、移动和删除已有代码	3
2.3	行号、高亮行和说明框	3
3	设置章节与自动目录	4
3.1	推荐的分类方法	4
3.2	目录是怎样生成的	4
4	在 Ubuntu 24.04.2 上准备环境	5
4.1	安装 Git、字体和 Typst	5
4.2	在本地预览	5
5	第一次上传到 GitHub	6
5.1	创建远程仓库	6
5.2	在 Ubuntu 上配置 SSH	6
6	在另一台设备上修改并同步	7
6.1	第一次取得仓库	7
6.2	完成一次修改	7
6.3	下载自动生成的 PDF	7
7	修改字体和字号	8
7.1	需要改的具体位置	8
7.2	把字体随仓库一起保存	8
7.3	调整字号后的检查	8
8	自动构建与版本发布	9
9	常见问题排查	10
9.1	Typst 提示找不到字体	10
9.2	Typst 提示找不到代码文件	10
9.3	PDF 没有出现刚修改的代码	10
9.4	长代码行越过边框	10
9.5	推送被拒绝	10
9.6	GitHub Actions 编译失败	10
10	一页式操作清单	11
10.1	日常添加算法	11
10.2	发布打印版	11
10.3	最终检查	11
11	官方参考资料	12

1 先认识这套模板

这套项目把“代码内容”和“打印样式”分开。平时最常改的是 `code/` 和 `main.typ`；只有需要调整字体、字号、页边距或代码框样式时，才改 `template.typ`。

<code>main.typ</code>	决定各章、小节、算法的顺序，并把代码文件插入 PDF。
<code>template.typ</code>	控制 A4 双栏、页眉、目录、行号、高亮行、字体和字号。
<code>code/</code>	保存真正参加比赛时要查阅的 C++ 代码片段。
<code>.github/workflows/build.yml</code>	每次上传后让 GitHub 自动编译 PDF；推送版本标签时自动发布。
<code>dist/</code>	本地编译得到的 PDF 输出目录。

1.1 最常用的 workflow

1

修改内容

编辑 `code/` 下的 C++ 文件；新算法还要在 `main.typ` 中登记章节和路径。

2

本地预览

运行下面的编译命令，打开 `dist/xcpc-codebook.pdf` 检查。

3

提交到 GitHub

用 Git 保存本次修改并上传。GitHub Actions 会再编译一遍，作为跨设备可下载的成果。

```
mkdir -p dist
typst compile --root . main.typ dist/xcpc-codebook.pdf
```

核心原则 每份算法代码只保留一个“源文件”。`main.typ` 只引用它，不再复制代码。这样修改一次，PDF 和仓库里的代码就会一起更新。

2 添加和修改代码片段

2.1 添加一个新算法：以 Boruvka 为例

Boruvka 属于“图论 → 生成树 → Boruvka”。推荐让文件夹结构也表达同样的分类：

```
code/
├─ graph/
│   └─ mst/
│       ├── kruskal.cpp
│       ├── prim.cpp
│       └─ boruvka.cpp
```

先创建并测试 `code/graph/mst/boruvka.cpp`，再在 `main.typ` 的图论大章中加入：

```
= 图论 / Graph

== 生成树 / Spanning Tree

=== Boruvka

#code-file("code/graph/mst/boruvka.cpp")
```

标题层级与 PDF 结构一一对应：

写法	含义	示例
<code>= 标题</code>	大章	图论
<code>== 标题</code>	小节	生成树
<code>=== 标题</code>	算法条目	Boruvka

不要重复建章 如果 `= 图论` 和 `== 生成树` 已经存在，只在该小节下增加 `=== Boruvka` 与 `#code-file(...)`。同一级标题出现两次，会在目录里变成两个分散的章节。

2.2 修改、移动和删除已有代码

- 只修改实现：直接编辑对应的 `.cpp` 文件。重新编译时，Typst 的 `read(...)` 会读取最新内容，不需要改 `main.typ`。
- 改算法名称或说明：修改 `main.typ` 中该算法上方的三级标题和文字。
- 移动或重命名文件：移动文件后，同时修改 `#code-file(...)` 的路径。Ubuntu 文件名区分大小写，`MST` 和 `mst` 不是同一路径。
- 删除算法：先从 `main.typ` 删除标题、说明和 `#code-file(...)`，再删除对应代码文件。

修改前后建议运行自己的样例或 Online Judge 测试。模板库中错误的代码通常比没有代码更危险。

2.3 行号、高亮行和说明框

普通插入：

```
#code-file("code/graph/mst/boruvka.cpp")
```

突出关键行（行号从 1 开始）：

```
#code-file(
  "code/graph/mst/boruvka.cpp",
  highlights: (8, 13, 21),
)
```

加入复杂度、前置条件或易错点：

```
#note-box[
  复杂度：$O(m \log n)$。注意图可能不连通。
]
```

排版建议 代码尽量控制在约 80 列以内；把超长表达式拆行，把调试输出和无关宏删掉。这样双栏打印时更清晰，也能减少长标识符越过代码框的情况。

3 设置章节与自动目录

3.1 推荐的分类方法

以“问题领域 → 算法族 → 具体模板”为主线，不建议按作者、比赛或添加日期分类。例如：



对应的 `main.typ` 骨架如下：

```

= 图论 / Graph

== 生成树 / Spanning Tree
=== Kruskal
#code-file("code/graph/mst/kruskal.cpp")

=== Prim
#code-file("code/graph/mst/prim.cpp")

=== Boruvka
#code-file("code/graph/mst/boruvka.cpp")

== 网络流 / Network Flow
=== Dinic
#code-file("code/graph/flow/dinic.cpp")
  
```

尽量不要从一级标题直接跳到三级标题。连续的层级既便于阅读，也能让 PDF 书签和辅助工具正确理解文档结构。

3.2 目录是怎样生成的

封面后的目录由 `template.typ` 中这行自动生成：

```
#outline(title: none, depth: 3, indent: 1em)
```

- `depth: 3`：目录显示大章、小节和具体算法。
- 改成 `depth: 2`：只显示大章和小节，适合算法很多的代码本。
- 改成 `depth: 4`：连四级标题也显示；通常会让目录过密。
- `indent: 1em`：控制子目录缩进。

标题编号由 `template.typ` 中的下面一行控制：

```
#set heading(numbering: "1.1")
```

如果某个标题只想在正文出现、不想进入目录，可显式写：

```
#heading(level: 3, outlined: false)[临时说明]
```

无需手写页码 新增、删除或移动章节后，只需重新编译。Typst 会自动重排页码、目录和 PDF 书签，不需要手工维护目录。

4 在 Ubuntu 24.04.2 上准备环境

4.1 安装 Git、字体和 Typst

先安装基础工具与开源中文字体：

```
sudo apt update
sudo apt install -y git curl xz-utils fonts-noto-cjk
```

Typst 官方提供 Linux 预编译包。下面以模板固定使用的 0.15.1 为例：

```
cd /tmp
curl -L0 https://github.com/typst/typst/releases/download/v0.15.1/typst-x86_64-unknown-linux-musl.tar.xz
tar -xf typst-x86_64-unknown-linux-musl.tar.xz
mkdir -p ~/.local/bin
cp typst-x86_64-unknown-linux-musl/typst ~/.local/bin/typst
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
typst --version
```

如果你已有 Rust 工具链，也可以按官方说明安装：

```
cargo install --locked typst-cli
```

版本建议 本地 Typst 与 `.github/workflows/build.yml` 中的版本保持一致，能减少“本地正常、GitHub 编译不同”的情况。以后升级时，两处一起改，并重新检查整本 PDF。

4.2 在本地预览

进入仓库根目录执行：

```
mkdir -p dist
typst compile --root . main.typ dist/xcpc-codebook.pdf
```

想在保存文件后自动重编译，可使用：

```
typst watch --root . main.typ dist/xcpc-codebook.pdf
```

停止监听按 `Ctrl+C`。即使本机暂时没有 Typst，也可以上传到 GitHub，让 Actions 代为编译；但赛前仍建议在本机打开 PDF 检查分页和字体。

5 第一次上传到 GitHub

5.1 创建远程仓库

在 GitHub 网页右上角选择 **New repository**，填写仓库名，例如 `xcpc-typst-codebook`。如果是把现有模板上传进去，创建时不要勾选 README、License 或 `.gitignore`，避免首次推送出现无关历史。

然后在项目根目录执行。下面的远程地址使用 SSH；如果本机尚未配置密钥，先完成 5.2，再回来执行 `remote` 和 `push`：

```
git init
git config user.name "YOUR_NAME"
git config user.email "YOUR_EMAIL"
git add .
git commit -m "init: add Typst codebook"
git branch -M main
git remote add origin git@github.com:YOUR_NAME/YOUR_REPO.git
git push -u origin main
```

把 `YOUR_NAME`、`YOUR_EMAIL` 和 `YOUR_REPO` 换成自己的信息。

5.2 在 Ubuntu 上配置 SSH

SSH 能避免每次推送都输入账号信息。生成密钥：

```
ssh-keygen -t ed25519 -C "YOUR_EMAIL"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

复制最后输出的整行，在 GitHub 打开 **Settings** → **SSH and GPG keys** → **New SSH key**，粘贴并保存。随后测试：

```
ssh -T git@github.com
```

密钥安全 只能上传或复制 `id_ed25519.pub`。不要把 `~/.ssh/id_ed25519` 私钥放进仓库、聊天记录或网盘。首次连接时应核对 GitHub 官方公布的 SSH 主机指纹。

6 在另一台设备上修改并同步

6.1 第一次取得仓库

在第二台 Ubuntu 设备上完成 SSH 设置后：

```
git clone git@github.com:YOUR_NAME/YOUR_REPO.git
cd YOUR_REPO
git status
```

之后每次开始编辑前，先同步远端：

```
git pull --ff-only
```

6.2 完成一次修改

以增加 Boruvka 为例：

```
git status
git add code/graph/mst/boruvka.cpp main.typ
git commit -m "feat: add Boruvka template"
git pull --rebase
git push
```

这里的顺序很重要：

1. `git status`：确认只包含自己想提交的文件。
2. `git add ...`：选择本次要提交的代码和章节清单。
3. `git commit`：在本地保存一个可追踪版本。
4. `git pull --rebase`：把另一台设备可能已上传的新提交接到前面。
5. `git push`：上传到 GitHub。

多设备习惯 “开始前 pull，结束后 push”。尽量不要让两台设备同时修改 `main.typ` 的同一段。若出现冲突，先阅读冲突标记并保留正确内容，再执行 `git add` 和 `git rebase --continue`；不要直接覆盖远端。

6.3 下载自动生成的 PDF

每次推送后，打开仓库的 **Actions** 页面，进入最新的成功任务，在页面底部下载 PDF artifact。推送版本标签时，工作流还会自动创建 GitHub Release：

```
git tag v1.0
git push origin v1.0
```

以后可用 `v1.1`、`v1.2` 等标签保存赛前稳定版本。标签应只打在已经打开检查过的 PDF 对应提交上。

7 修改字体和字号

7.1 需要改的具体位置

所有主要字体设置都在 `template.typ`：

要修改的内容	搜索位置
正文中英文字体	搜索 <code>#set text(</code> 内的 <code>font:</code> 。
正文基础字号	同一处的 <code>size: 7.2pt</code> 。
代码字体与字号	搜索 <code>show raw: set text(</code> ；当前为 <code>DejaVu Sans Mono</code> 和 <code>6.15pt</code> 。
章、小节标题字号	搜索 <code>heading.where(level: 1)</code> 、 <code>level: 2</code> 和 <code>level: 3</code> 。
封面标题字号	搜索 <code>cover</code> 附近的 <code>text(size: ...)</code> 。

正文设置中的第一个字体负责拉丁字母，第二个字体负责中文。若想改为思源宋体和 JetBrains Mono，可直接替换为：

```
#set text(  
  font: (  
    (name: "Libertinus Serif", covers: "latin-in-cjk"),  
    "Source Han Serif SC",  
  ),  
  size: 7.2pt,  
)  
  
#show raw: set text(  
  font: "JetBrains Mono",  
  size: 6.15pt,  
)
```

字体名称必须与字体内部记录的 family name 完全一致，不一定等于文件名。可用下面的命令核对：

```
typst fonts | grep -E "Noto|Source Han|JetBrains|DejaVu"
```

7.2 把字体随仓库一起保存

为了让 Windows、Ubuntu 和 GitHub Actions 的输出一致，最稳妥的方式是把允许再分发的字体文件放入 `fonts/`，并保留字体许可证：

```
fonts/  
├─ SourceHanSerifSC-Regular.otf  
├─ JetBrainsMono-Regular.ttf  
└─ LICENSES/
```

本地编译命令改成：

```
typst compile --root . --font-path fonts main.typ dist/xcpc-codebook.pdf
```

同时把 `.github/workflows/build.yml` 中的编译命令也加上 `--font-path fonts`。否则本地能找到字体，GitHub Actions 可能找不到。

字体选择建议 优先使用 Noto、思源、Libertinus、DejaVu、JetBrains Mono 等可在 Linux 使用且许可证清晰的字体。不要依赖微软雅黑、宋体、Consolas 等只在某台 Windows 设备上存在的字体。

7.3 调整字号后的检查

代码字号每改变 `0.2pt`，都可能影响整本分页。修改后至少检查：

- 最长的代码行有没有越过右边框；
- 目录是否挤到额外页面；
- 页眉是否与正文重叠；
- 中文、数字、运算符是否来自协调的字体；
- 打印为实际大小时，行号与注释能否清楚辨认。

8 自动构建与版本发布

仓库自带 `.github/workflows/build.yml`。它通常完成三件事：

1. 安装中文字体和固定版本的 Typst；
2. 编译 `main.typ` 并上传 PDF artifact；
3. 当推送 `v*` 标签时，把 PDF 附加到 GitHub Release。

普通修改只需 `git push`。准备一份稳定的比赛打印版时：

```
typst compile --root . main.typ dist/xcpc-codebook.pdf
# 打开 PDF，检查目录、代码框、页码和最后一页

git status
git add .
git commit -m "release: prepare v1.0 codebook"
git push
git tag v1.0
git push origin v1.0
```

赛前建议 Release 是归档，不是测试环境。先在普通提交对应的 Actions artifact 中检查 PDF，再打标签。比赛规则若限制页数、单双面或字体大小，还需按当场规则复核。

9 常见问题排查

9.1 Typst 提示找不到字体

运行 `typst fonts` 查看实际名称；安装字体后重新打开终端。若字体在仓库 `fonts/` 中，确认本地命令和 Actions 命令都含 `--font-path fonts`。

9.2 Typst 提示找不到代码文件

检查 `#code-file("../")` 的路径、扩展名和大小写。命令必须从仓库根目录执行，并保留 `--root .`。Ubuntu 会严格区分 `Boruvka.cpp` 与 `boruvka.cpp`。

9.3 PDF 没有出现刚修改的代码

确认文件已保存；确认 `main.typ` 引用的是正在编辑的那一份；删除旧 PDF 后重新编译也可帮助判断是否看错了输出文件。

9.4 长代码行越过边框

优先拆分长表达式、缩短局部变量名或将注释放到上一行。仍然过长时，再小幅降低 `template.typ` 中代码字号。不要一开始就把整本代码缩得难以阅读。

9.5 推送被拒绝

远端通常已有更新。先执行：

```
git pull --rebase
git push
```

如果 Git 报冲突，打开带有 `<<<<<<<`、`=====` 和 `>>>>>>>` 的文件，人工合并后：

```
git add main.typ
git rebase --continue
git push
```

9.6 GitHub Actions 编译失败

打开失败任务，先看第一条红色错误。常见原因是文件路径大小写错误、字体缺失、Typst 版本变化或工作流里的输出目录不存在。本地使用与工作流相同的命令复现，通常最容易定位。

10 一页式操作清单

10.1 日常添加算法

```
# 开始前同步
git pull --ff-only

# 新增代码，并在 main.typ 登记章节
mkdir -p code/graph/mst
$EDITOR code/graph/mst/boruvka.cpp
$EDITOR main.typ

# 本地检查
typst compile --root . main.typ dist/xcpc-codebook.pdf

# 保存并上传
git status
git add code/graph/mst/boruvka.cpp main.typ
git commit -m "feat: add Boruvka template"
git pull --rebase
git push
```

10.2 发布打印版

```
# 检查最新 Actions artifact 与本地 PDF
git tag v1.0
git push origin v1.0

# 在 GitHub Releases 页面下载最终 PDF
```

10.3 最终检查

- 目录中的章、小节、算法顺序正确；
- 新增代码已经过编译与样例测试；
- 所有 `#code-file` 路径在 Ubuntu 上大小写一致；
- 代码没有越过边框，字体打印后可读；
- Git 工作区干净，最新修改已经推送；
- 打标签前已打开并逐页检查 PDF。

11 官方参考资料

以下页面用于核对安装、排版和 GitHub 操作；命令或版本变化时，以官方文档为准。

- [Typst 官方仓库与安装说明](#)
- [Typst：读取外部文件](#)
- [Typst：原始文本与代码高亮](#)
- [Typst：标题层级](#)
- [Typst：自动目录](#)
- [Typst：字体与文本设置](#)
- [GitHub：创建仓库](#)
- [GitHub：把本地项目上传到仓库](#)
- [GitHub：Git 基础与常用操作](#)
- [GitHub：添加 SSH 公钥](#)
- [GitHub：SSH 主机指纹](#)

手册结束 · 建议将本 PDF 与代码本 Release 一起保存到比赛准备清单